

Programación declarativa

• Haskell •

Doble grado en Ingeniería Informática y Matemáticas

Universidad Complutense de Madrid

Óscar Martín — Curso 2026/27

7. Tipos y clases de tipos básicos

Tipos

- Haskell es estáticamente tipado.
- Haskell es *fuertemente* tipado.
- Pero el tipo puede ser polimórfico.
 - Cada expresión en Haskell tiene un tipo *más general*.
- El compilador de Haskell es capaz de inferir los tipos de las expresiones que escribamos.
 - `:t not False ~> not False :: Bool`
 - Pero es conveniente y es costumbre escribirlos.

```
esCapicua :: [Int] -> Bool
esCapicua xs = reverse xs == xs
```

```
n :: Int
n = 5
m = 5 :: Int
```

Algunos tipos básicos de Haskell

- `Bool` (contiene los valores `True` y `False`)
- `Int` (al menos 30 bits, 64 en GHC), `Integer` (sin límite de tamaño)
- `Float`, `Double`
- `Char` (entre comillas simples)
- `String` (= `[Char]`) (entre comillas dobles)
- Veremos más adelante cómo podemos definir nuevos tipos.

Clases de tipos

- `esCapicua :: Eq a => [a] -> Bool`
 - `Eq` es una clase de tipos.
 - `Eq a` es una *restricción de clase* (o *restricción de tipo*).
 - `Eq a => [a] -> Bool` $\equiv$ “para cualquier tipo que cumpla lo que requiere la clase `Eq`...”
 - La misma definición, `esCapicua xs = reverse xs == xs`, sirve para todos los casos.
- Prueba `:i Eq`.

```
...
class Eq a where
    (==) :: a -> a -> Bool
    (/=) :: a -> a -> Bool
...
```

- Ojo: Las clases de tipos de Haskell no son lo mismo que las clases de OOP.
 - Si acaso, se parecen más a los *interfaces*.

Algunas clases de tipos de Haskell

- `Eq`
 - define `==` y `/=`
 - `Bool`, `Char`, ...
 - `Eq a => Eq [a]`, ...
 - En cambio, los tipos de funciones (`a -> b`) no están en `Eq`.
- `Ord`
 - extiende `Eq`
 - define `<`, `<=`, ...
 - `Bool`, `String`, ...
 - `Ord a => Ord [a]`, ...
- `Num`
 - define `+`, `*` y otros
 - Los literales enteros pertenecen a todos los tipos de esta clase.
 - `Int`, `Float`, ...
- `Integral`
 - extiende `Num`
 - define `mod`, `div`, ...
 - `Int`, `Integer`, ...
- `Fractional`
 - extiende `Num`
 - define `/`
 - Los literales con decimales pertenecen a todos los tipos de esta clase.
 - `Float`, `Double`, ...

Ejemplos de funciones con restricciones de clase

```
maximo :: Ord a => a -> a -> a
maximo x y = if x < y then y else x

tresIguales :: Eq a => a -> a -> a -> Bool
tresIguales x y z = x == y && y == z

repite :: (Num a, Ord a) => a -> b -> [b]
-- repite 3 7 = [7, 7, 7]
repite n x = if n <= 0 then []
              else x : repite (n-1) x

signo :: (Ord a, Num a, Num b) => a -> b
signo n = if n < 0 then -1 else
          if n == 0 then 0 else 1
```

- Puede haber varias restricciones de clase en una función.
- Podemos definir nuevas clases de tipos.
- Podemos decir qué tipo concreto queremos que Haskell dé a una expresión:
 - `4 :: Float`; `(x::T)+y`, ...

Ejercicios

1. Determina los tipos de estas expresiones:

```
| x1 = ["a", ['b', 'c']]  
| x2 = [init, reverse, id]
```

2. Determina los tipos más generales de estas funciones:

```
| intercambia (x, y) = (1, x)  
| tieneUno xs = length xs == 1  
| esListaVacía xs = xs == []  
| aplicaAplica f x = f (f x)  
| rara x y = (x==y, y)  
| rara2 x = rara 2 x
```

3. Existen en Haskell dos clases de tipos llamadas `Enum` y `Bounded`. Averigua qué son, qué funciones incluyen y qué tipos pertenecen a cada una. Prueba en ghci `succ 'a'`, `succ False` y `succ True`. Define una función

```
| siguiente :: (Enum a, Bounded a, Eq a) => a -> a
```

que devuelva el sucesor de su argumento, salvo para el último valor del tipo, para el que devuelve el primero. Es decir,

- siguiente 'a' $\rightsquigarrow$ 'b'
- siguiente False $\rightsquigarrow$ True
- siguiente True $\rightsquigarrow$ False
- siguiente (maxBound :: Int) $\rightsquigarrow$ -9223372036854775808
- siguiente GT $\rightsquigarrow$ LT (El tipo `Ordering` consta de los valores `LT < EQ < GT`.)

¿Hacen falta las tres restricciones de clase?